

Pioneer.jl — Optimization & Cleanup Opportunities

with a detailed plan for the Scoring working-set reduction

2026-06-25 | surveyed against `origin/develop`

Scope. A survey of optimization / cleanup opportunities across five threads (performance, feature reduction, quant, organization, PackageCompiler), followed by a **detailed, sequenced implementation plan for Thread 2 — cutting the ~4.3 GB Scoring working set** that dominates peak RSS. PackageCompiler is the other high-value target (quick blockers, removes the ~4.5 GB JIT overhead for end users) and is summarized at the end.

1 Opportunity survey

1. PackageCompiler binary (high ROI — quick blockers)

A relocatable app pipeline already exists (`.github/workflows/build_app_{linux, macos, windows}.yaml` + `src/build/snoop.jl`). It fails on mundane blockers:

- **Julia version mismatch** — `Project.toml` pins `julia="1.12.5"` but all three workflows install `1.11` $\Rightarrow$ `Pkg.instantiate()` rejects before compiling. (~15 min fix.)
- **Missing entry point** — the `executables` list references `main_ParseSpecLib`, which is not defined. `create_app` errors. (~10 min.)
- **LightGBM relocatability** — “`lib_lightgbm found in system dirs!`” means it binds a host lib, not the bundled `LightGBM_jll` artifact. Latent hazard; half-day of build-iterate to pin to the JLL.

Verdict: the two config fixes are an hour; a working *sysimage* is reachable in a couple hours; a verified relocatable *app* ~1 day.

2. The 4.3 GB Scoring working set (this document’s focus — see §2)

The live-heap high-water-mark we measured (~7.5 GB) is ~3.2 GB library + ~4.3 GB scoring buffers. The 4.3 GB is reducible (detailed plan below).

3. Other performance / memory (not yet touched)

Opportunity	Win	Effort
Per-thread duplicated read-only Set of all precursors, rebuilt per file (<code>Integrate.../utils.jl:658</code> , <code>HuberTuning/utils.jl:79</code>) — scales with <code>#threads</code> (~97% waste at 32 threads)	M–L mem	S
Float64 score buffers in LGBM CV downcast to Float32 anyway (<code>MainSearch/scoring.jl:136--231</code>)	~0.2 GB/vec	S [†]
grouped_catalog_cache / species_agg rebuilt per file (<code>protein_inference_pipeline.jl:1278</code> , <code>maxLFQ.jl:778</code>) — same dedup pattern as the wins already landed	S–M mem	S–M
<code>vcut(parts...)</code> 2× transient peak (<code>mbr_streaming.jl:464</code> , <code>PrecursorScoringSearch.jl:249</code>)	S mem	S

[†] **not** bit-identical — q-value/PEP computed from Float32 vs Float64 scores can shift IDs marginally; treat as an approximation change.

4. Feature reduction (your hypothesis — partly right)

The flanking features don’t make MainSearch columns *redundant*; rather there is **independent dead weight** stored per-PSM across ~14M rows:

- **Dead scalar columns** in MainSearchScoredPSM absent from every final model list: `matched_ratio`, `max_matched_residual`, `max_unmatched_residual`, `total_ions_iso` (~112 MB/file, near-zero risk; `b_count/best_rank` after a probit-path confirm).
- **The big one:** `fitted_frag1..8_int` + `shadow_frag1..8_int` (16 cols) are stored on the full 14M-row table but consumed only on the ~400k best-per-precursor rows (~900 MB/file). *Caveat:* the consumer (`_mainsearch_radius1.2d_shadow_hellinger`) uses a **radius-1** window, so it needs best rows *plus their adjacent scans* — see §2 Stage 2.

The team already removed similar dead features (commit `ac9e0ea12`, a clean 5-site pattern); these reductions have been ID-neutral in prior sweeps.

5. Organization / code-health

- **Dead MS1** `build_chromatograms` block-comment (~250 LOC, `Integrate.../utils.jl:892--1143`) referencing soon-to-be-deleted APIs — clean delete (S).
- Oversized modules mixing concerns (`features.jl` 1613, `scoring.jl` 1477, `QuadTuning/utils.jl` 1372 blends plotting+logic) — mechanical splits (M each).
- **Risk flag:** the big feature/scoring math (`features.jl`, `scoring.jl`, `ParameterTuning`, `QuadTuning` collection) has **no unit tests** — correctness rides on one ecoli end-to-end fixture. Worth targeted unit tests *before* feature-reduction work.

Quant ideas

The structural/perf explorations did not surface new quant *algorithms* (out of scope); prior work already covers Huber / smoothing / group-lasso extensively. Quant innovation is a separate, open-ended R&D track — recommend treating it deliberately, not bundled here.

2 Detailed plan — cut the 4.3 GB Scoring working set

Goal. Reduce the live-heap high-water-mark during Precursor Scoring (and the upstream MainSearch table feeding it) so a run fits a lower RSS ceiling, *without changing results*. Sequenced safest→biggest. Each stage is independently shippable and verified before the next.

Verification methodology (applies to every stage)

- **Bit-identical class** (dead columns, shared Sets, free-after-write, fitted/shadow restructuring *if neighbors preserved*): run 1–2 file MTAC before/after, byte-compare `precursors_long.arrow` + `protein_groups_long.arrow` (the harness we already use). Must be identical.
- **ID-equivalent class** (no-MBR OOM reroute, any `Float32` buffer change): scores may differ in the last bits, so byte-compare can fail legitimately. Instead compare **ID counts at 1% FDR** (precursor + protein-group) and the entrapment-EFDR, on MTAC *and* Exploris; require no regression. Keep the change behind a flag until confirmed.
- Always re-check `origin/feature/reduce-allocations` / develop for conflicts before each commit (these files are actively edited).

Stage 0 — warm-ups (S, bit-identical, ship this week)

1. **Shared precursor Set.** `Integrate.../utils.jl:658` and `HuberTuning/utils.jl:79` build `[Set(psms[:,precursor_idx]) for _ in 1:nthreads()]` — one full Set *per thread*, rebuilt per file. Verified read-only (membership only). **Build one Set before the spawn loop, pass the shared instance to all tasks.** Win scales with thread count (biggest where peak RSS is worst). Bit-identical.
2. **Free best_psms in the no-MBR path.** The MBR path does `psms = DataFrame(); GC.gc()` after writing `(score_psms.jl:343)`; the no-MBR path (`_score_precursor_isotope_traces_no_mbr`) returns without freeing the full-experiment `best_psms` after `write_scored_psms_to_files!` (~line 395). Add the same free. Removes one full table from the peak.
3. **Drop 4 dead scalar columns** from `MainSearchScoredPSM (ScoredPSMs.jl)`: `matched_ratio`, `max_matched_residual`, `max_unmatched_residual`, `total_ions_iso`. Remove from (a) the struct, (b) the `Score!` constructor call, (c) any `_COLUMNS` tuple — the `ac9e0ea12` 5-site pattern. ~112 MB/file off the 14M-row table. Bit-identical (zero consumers); confirm with a grep for each name first. (`b_count`, `best_rank` as a follow-up after confirming no probit/MBR fallback reads them.)

Stage 1 — no-MBR streaming reroute (M, the biggest single memory win)

Problem. `_score_precursor_isotope_traces_no_mbr` calls `load_psms_for_lightgbm`, which does `DataFrame(Tables.co` — it materializes the *entire experiment-wide PSM set* into one in-memory `DataFrame`, detached from the mmap. This single object is the bulk of the 4.3 GB on non-MBR runs.

Solution. The MBR path already streams via `train_and_predict_pass1_oom!` (`pass1_oom.jl:436`), which is **MBR-agnostic** — it trains Pass-1 on whatever feature set it's given and writes per-file `.pass1_sidecar.arrow` (OOF `trace_prob_prepass`). Route the no-MBR path through it:

1. Call `train_and_predict_pass1_oom!(file_paths, precursors; features = ADVANCED_FEATURE_SET \ MBR-features...)` instead of the in-memory `load+train`.

2. For no-MBR, **skip** the MBR/recovery/FTR steps entirely; set `trace_prob = trace_prob_prepass` and `mbr_recovered = false` (mirroring the current no-MBR semantics) inside the per-file sidecar, then merge the Pass-1 sidecars into each main file (a simpler merge than `merge_mbr_sidecars_into_main!` — only the Pass-1 columns).
3. Keep `_score_precursor_isotope_traces_no_mbr` as a fallback for tiny runs (the OOM path has per-file overhead); dispatch on a size threshold or a flag.

Risk / verification. Streaming/fold-ordering can perturb scores in the last bits $\Rightarrow$ *ID-equivalent* class, not bit-identical. Gate behind a flag (`force_oom` already exists at `score_psms.jl:28`); compare 1%-FDR ID counts + EFDR on MTAC and Exploris (non-MBR config) before/after. **Feasibility: confirmed** — the trainer is already decoupled; the work is the simpler no-MBR sidecar merge plumbing.

Stage 2 — fitted/shadow frag restructuring (M–L, ~900 MB/file)

Problem. The 16 `fitted_frag*_int/shadow_frag*_int` columns (`ScoredPSMs.jl`) are written for *every* of the 14M deconvolved PSM rows (`process_scans_fused.jl:157` materializes the whole struct), but are consumed only at `scoring.jl:996--1194` to compute `smoothed_2d_shadow_hellinger` on the ~400k best-per-precursor rows, then dropped (`select!(... Not(...))`, `scoring.jl:1216`).

Key caveat (do not skip). The consumer `mainsearch_radius1_2d_shadow_hellinger` uses a **radius-1** window — for each best PSM it reads the fitted/shadow intensities of that PSM *and its ± 1 adjacent scans*. So you cannot keep only the 400k best rows; you must preserve each best PSM’s immediate chromatographic neighbors.

Approach (behavior-preserving).

1. Keep the 16 columns on the *full* table through `select_best_per_precursor!` only long enough to compute the radius-1 hellinger *there* (where the neighbor rows are still present), then drop them **before** the table is widened with the rest of the scoring features — i.e. move the `smoothed_2d_shadow_hellinger` computation earlier and the `select!(Not(...))` drop right after it. This shortens the lifetime of the 16 columns from “whole scoring phase” to “one pass”, cutting their peak contribution without changing the result. (*Lower-risk first step.*)
2. (*Optional, larger*) Avoid storing them in `MainSearchScoredPSM` at all: capture fitted/shadow intensities into a compact side buffer keyed by (precursor, scan) only for scans within radius-1 of a best PSM, computed during best-PSM selection. Higher effort, higher risk — defer until Step 1 is measured.

Risk / verification. Bit-identical *iff* the same rows (best ± 1) feed the hellinger; verify by byte-compare. Step 1 is the safe, most of the win; Step 2 only if needed.

Sequencing & expected payoff

Stage	Item	Est. win	Class
0	shared Set / free / 4 dead columns	0.1–0.5 GB + per-thread	bit-identical
1	no-MBR OOM streaming reroute	bulk of 4.3 GB (non-MBR)	ID-equivalent
2 (step 1)	fitted/shadow lifetime shortening	~0.9 GB/file peak	bit-identical
2 (step 2)	fitted/shadow side-buffer (optional)	more	bit-identical (verify)

Ship Stage 0 immediately (low risk, includes the thread-scaling Set win). Stage 1 is the headline memory reduction but needs the ID-equivalent A/B and the no-MBR merge plumbing. Stage 2 step 1 is a safe, high-value lifetime change; step 2 only if the ceiling still isn't met. Re-measure peak RSS (the single-thread + multi-thread harness already built) after Stage 1 and Stage 2 to confirm the high-water-mark actually drops.